Neuron Architecture

Hybrid execution model for the workflow engine: plans are generated and orchestrated server-side, but task execution can be routed to external processes ("neurons") — including desktop clients holding user-specific MCP connections — or to in-cluster workers, based on declared capabilities and scope.

Motivation

The baseline architecture (see `ARCHITECTURE.md`) executes all tasks in server-side worker processes. This works for generic compute (scripts, HTTP, LLM calls) but breaks for user-specific integrations:

- **MCP connections are per-user.** A desktop client authenticated to Google Drive as user `abc` cannot legally (or safely) process a Drive task on behalf of user `xyz`.
- **Capability availability varies by client.** A desktop binary ships with different MCP servers than another; a web client has none.
- **Trust boundaries differ.** A server-side worker has database credentials and service-account trust; a user-installed desktop neuron does not.

The neuron architecture introduces a routing layer that matches tasks to executors by capability, scope, and authorization, while keeping plan generation and orchestration centralized.

Core Concepts

- **Neuron** — any process that executes tasks. Identified by an authenticated identity (JWT from Moku), advertises one or more capabilities at registration time. The built-in `apps/worker` is itself a neuron; external neurons (desktop, per-tenant agents) connect over the wire protocol.
- **Capability** — a typed function signature that a neuron can execute. Identified by a namespaced type string (`core/script.python`, `desktop/mcp/drive.read`). Schemas for input/output are not global; they travel with the plan (see *Plan-Time vs Registration-Time Schemas*).
- **Scope** — who a capability advertisement is valid for. Either `any` (any user's task) or `{ onBehalfOf: userId }` (only that user's tasks). Scope is enforced by the gateway at both registration time and routing time.
- **Gateway** — the server-side component that accepts neuron connections, authorizes capability advertisements, binds consumer channels to per-user queues, and forwards work. Mounted as routes on `apps/api` for v1, isolated enough to extract into its own service if scale demands.
- **On-behalf-of** — the user identity a task executes for. Set at plan time (from the workflow's owner), stamped on every task, used to select the target queue.

System Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                   CLIENTS                                    │
│                                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │ Desktop App  │  │    Web UI    │  │   CLI Tool   │  │   REST API   │      │
│  │ (is also a   │  │              │  │              │  │              │      │
│  │   neuron)    │  │              │  │              │  │              │      │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘      │
└─────────┼─────────────────┼─────────────────┼─────────────────┼──────────────┘
          │ plan req        │                 │                 │
          │ SSE+POST        │                 │                 │
          ▼ (neuron conn)   ▼                 ▼                 ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                                 APPS/API                                     │
│                                                                              │
│  ┌─────────────┐  ┌─────────────┐  ┌─────────────┐  ┌──────────────────────┐ │
│  │ Auth (JWT)  │  │  Workflow   │  │ Orchestrator│  │    Neuron Gateway    │ │
│  │ via Moku    │  │  Endpoints  │  │   (/plan)   │  │   (SSE + POST)       │ │
│  │    JWKS     │  │             │  │             │  │  ┌────────────────┐  │ │
│  └─────────────┘  └─────────────┘  └─────────────┘  │  │ Session Reg    │  │ │
│                                                     │  │ Capability Auth│  │ │
│                                                     │  │ Queue Lifecycle│  │ │
│                                                     │  └────────────────┘  │ │
│                                                     └──────────────────────┘ │
└────────┬─────────────────────────┬─────────────────────────┬─────────────────┘
         │                         │                         │
         ▼                         ▼                         ▼
┌─────────────────────┐  ┌─────────────────────┐  ┌──────────────────────────┐
│  PostgreSQL         │  │      RabbitMQ       │  │      Moku API            │
│  (BigBrain schema)  │  │                     │  │  (platform services)     │
│  ┌───────────────┐  │  │  ┌───────────────┐  │  │  ┌────────────────────┐  │
│  │ workflows     │  │  │  │ tasks.any.{cap}│ │  │  │ JWT / JWKS         │  │
│  │ plans         │  │  │  ├───────────────┤  │  │  │ User profile       │  │
│  │ tasks         │  │  │  │ tasks.user.{U}.│ │  │  │ Org membership     │  │
│  │ neuron_sessions│ │  │  │   {capability} │ │  │  │ Policy config      │  │
│  │ policy_audit  │  │  │  │  (per-user, ...│ │  │  │ (cross-service)    │  │
│  └───────────────┘  │  │  │   lazy-created)│ │  │  └────────────────────┘  │
│    (Moku reads for  │  │  └───────────────┘  │  │                          │
│     reporting, does │  │  ┌───────────────┐  │  │                          │
│     not write)      │  │  │ task.results  │  │  │                          │
│                     │  │  │ (single-active)│ │  │                          │
│                     │  │  └───────────────┘  │  │                          │
└─────────────────────┘  └─────────────────────┘  └──────────────────────────┘
                                   ▲
```

```
                                    │ lease / ack
                                    │
┌───────────────────────────────────┼───────────────────────────────────────────────┐
│                                NEURONS                                            │
│   ┌───────────────────────────────┴─────┐   ┌─────────────────────────────┐       │
│   │        Built-in Neuron              │   │    External Neuron          │       │
│   │      (apps/worker, in-cluster)      │   │    (desktop, on-prem)       │       │
│   │                                     │   │                             │       │
│   │  advertises: core/* capabilities    │   │  advertises:                │       │
│   │  scope: any                         │   │    desktop/mcp/*            │       │
│   │  uses @workflow/neuron-sdk          │   │    scope: onBehalfOfAuth    │       │
│   │                                     │   │  uses @workflow/neuron-sdk  │       │
│   └─────────────────────────────────────┘   └─────────────────────────────┘       │
└───────────────────────────────────────────────────────────────────────────────────┘
```

Execution Model

Registration

A neuron opens an SSE stream to `/neuron/events` (server → neuron) and sends a `register` frame via `POST /neuron/register` with its JWT in the `Authorization` header. The frame carries:

- `neuronId` — stable id per device/process
- `capabilities[]` — each with `type`, `scope`, `concurrency`

The gateway:

1. Verifies the JWT via cached Moku JWKS.
2. For each advertised capability, checks the **advertise-leg authorization** — target model: org-level membership in Moku's capability catalog (HOL-2539); today: the `namespace_authorization` table, bypassable via `NEURON_NAMESPACE_AUTH_BYPASS`. See `NEURON_AUTH_SPEC.md` §2.
3. Invokes `PolicyEngine.checkRegistration()` (see *Policy Integration*).
4. For user-scoped advertisements, verifies the auth token is authorized for that user.
5. Lazily declares the matching RabbitMQ queues and opens consumer channels with `prefetch = concurrency`.
6. Persists the session to `neuron_sessions` for UI visibility and routing lookups.

Registration is the **routing concern** only. No schemas are exchanged at registration time — schemas live with each plan.

> **Authentication model (settled 2026-07-14 — full contract in `NEURON_AUTH_SPEC.md`).** Moku is the sole identity authority; BigBrain issues nothing and verifies Moku RS256 JWTs via cached JWKS. Desktop neurons present a user token obtained via Exchange Code login + RFC 8693 exchange (`aud=bigbrain`, performed in the Electron main process). Enterprise/headless neurons enroll once via Moku's RFC 8628 device flow (admin-approved, org bound to the approver), then mint short-lived `aud=bigbrain` tokens on

> demand via an OAuth2 `client_credentials` grant — the long-lived artifact is a revocable client credential in Moku, never a bearer JWT. The static `CORE_NEURON_JWT` service token is being retired on this path (HOL-397). De-authorization is push-first: Moku signals BigBrain to kill a revoked client's sessions; sessions are otherwise long-lived and never cycled on token expiry.

Capability Scoping

Scope is a property of each capability advertisement, not the neuron as a whole. One neuron may advertise mixed-scope capabilities:

```
{
  "capabilities": [
    { "type": "desktop/mcp/drive.read", "scope": { "onBehalfOf": "user:abc" }, "concurrency":
    { "type": "desktop/mcp/slack.post", "scope": { "onBehalfOf": "user:abc" }, "concurrency":
    { "type": "core/script.python",     "scope": "any",                        "concurrency":
  ]
}
```

The auth token's user binding is the **only** authority for `onBehalfOf` scope — a neuron cannot advertise a scope it is not authorized for; registration is rejected.

Per-User Queues

Queues for user-scoped capabilities are **lazily declared per (user, capability)** pair:

- Name: `tasks.{orgId}.user.{userId}.{capabilityType}` (e.g., `tasks.org1.user.abc.desktop.mcp.drive.read`) — org-partitioned (HOL-356); the orgId segment derives from the verified JWT, never from the wire
- `durable: true`
- `x-expires: 7d` — broker auto-deletes queue after 7d with no consumers and no messages
- `x-message-ttl: 24h` — individual tasks expire if unclaimed past this window
- DLX → `tasks.expired` — exposes timed-out tasks to the planner for re-plan or fail decisions

Queues for scope-`any` capabilities use a shared queue per (org, capability type): `tasks.{orgId}.any.{capabilityType}` .

Leasing and Restart Semantics

Tasks are leased, not dequeued. A neuron consumes a message without acking; the gateway writes the task's DB row to `assigned` with a lease expiry. The ack is sent only on terminal success. On disconnect, the broker redelivers the message automatically; a reaper flips stale `assigned` rows back to `pending` and the task resumes on the next consumer (possibly the same user's different device).

Consequence for handler authors: handlers must be idempotent, or use capability-native idempotency keys (e.g., Google Drive `requestId`). Restart semantics mean "a task that completed a side effect but failed to ack will re-run." This is a per-handler contract, not a framework-level guarantee.

Concurrency

Each capability advertisement carries `concurrency`. The gateway translates this into AMQP `basic.qos prefetch_count` on the consumer channel, so the broker handles backpressure — once the neuron has N unacked messages, it stops receiving new ones until acks flow.

Neurons may send `UPDATE_CAPABILITY` frames to change concurrency mid-session (e.g., desktop plugged in); the gateway applies new prefetch on next delivery without disturbing in-flight leases.

Heartbeats

Every ~10s, the neuron sends:

```
{ "type": "heartbeat", "neuronId": "...", "inFlight": [
    { "leaseId": "...", "taskId": "...", "startedAt": "..." }, ...
]}
```

Heartbeats (a) renew all active lease expiries in one round-trip, (b) let the gateway detect wedged neurons (inFlight full + no progress), (c) power the UI's "3/4 busy" display.

Plan-Time vs Registration-Time Schemas

Capabilities are named routing addresses; **their schemas are not globally registered**. Instead:

- The client initiating a workflow owns the handler catalog — the set of capability types available in its context, each with input/output schemas (zod → JSON Schema). The desktop knows its MCP tools; the server knows its bundled handlers.
- At plan creation, the client posts the catalog alongside the goal:

```
POST /plan
{
  "goal": "summarize the sales deck in Drive and post to #launches",
  "onBehalfOf": "user:abc",
  "handlers": [
    { "type": "desktop/mcp/drive.read", "input": {...}, "output": {...} },
    { "type": "core/llm.summarize",    "input": {...}, "output": {...} },
    { "type": "desktop/mcp/slack.post",  "input": {...}, "output": {...} }
  ]
}
```

- The LLM planner receives the catalog in its prompt context and emits a typed plan (dot-path bindings from context into each step's input).
- The planner validates its own plan against the catalog before persisting (type-compatibility check on every binding).
- The catalog is **snapshotted with the plan** — replay, debugging, re-execution use the schemas that were valid at plan time, not whatever is current.

Execution-time validation happens in the neuron SDK (input-schema check before invoking the handler, output-schema check before acking). Schema mismatches → specific nack codes; the orchestrator surfaces them to the user with a clear "plan expected shape X, neuron produced shape Y" error.

Capability Namespacing and Authorization

Capability types are namespaced: `{owner}/{package}/{name}` — e.g. `core/script.python`, `desktop/mcp/drive.read`, `org/{orgId}/erp.read`. The namespace is a **naming and routing convention**: it keeps capability types collision-free across producers and makes ownership legible.

Authorization of what a neuron may *advertise* is deliberately coarse and org-level (settled 2026-07-15; `NEURON_AUTH_SPEC.md` §2): **admission-on-advertisement**. The gateway upserts every advertised type into the org's admission ledger (`org_capabilities.status`); never-seen types are created per the org's `capability_admission` setting (`auto_approve` default → binds immediately; `require_approval` → `requested`, rejected with `CAPABILITY_PENDING_APPROVAL` until an admin approves in Moku), and `denied` rows never auto-resurrect. There are no per-neuron capability subsets and no per-identity namespace grants — any enrolled neuron of an org may serve any org-approved capability, with the blast radius bounded by org-partitioned queues. The earlier per-identity `namespace_authorization` table is retired (HOL-2539); the same ledger decides `register`, `update-capability`, and the planner's client-supplied catalogs.

User scope is authorized separately and unchanged: a capability advertised `onBehalfOf:userId` requires the presenting token's `sub` to match — the JWT is the only authority for user scope (`on_behalf_of` scope requirement).

Capability squatting (a malicious neuron claiming `core/*` addresses and siphoning tasks) is contained by the combination: connect requires an enrolled org-bound client, advertisement requires the type to be org-cataloged, and queues are org-partitioned — a rogue advertiser can at most receive its own org's traffic for capabilities its org admin has allowed. Rejections happen visibly at registration, not as silent routing failures at runtime.

Wire Protocol

Transport: **SSE for server → neuron + HTTPS POST for neuron → server**, with correlation ids linking responses. Enterprise-friendly (traverses proxies cleanly, looks like normal HTTPS), matches the transport style of OpenAI/Anthropic streaming APIs and MCP's HTTP surface.

Message Types (overview — full schema in `packages/neuron-protocol`)

Neuron → Server (POST):

- `register` — open session with capabilities
- `update-capability` — change concurrency or scope for an existing capability
- `heartbeat` — lease renewal + in-flight state
- `ack` — task completed successfully, with output

- `nack` — task failed (retryable | terminal) with reason
- `progress` — in-progress status update for UI

Server → Neuron (SSE):

- `registered` — registration confirmed, includes effective capability bindings after policy/auth filters
- `lease` — a task to execute (`leaseId`, task payload with input schema)
- `cancel` — cancel a specific lease (triggered by workflow cancellation)
- `capability-revoked` — a capability was deauthorized mid-session; in-flight leases on that capability must fail
- `disconnect` — graceful close from server

SSE resumption uses `Last-Event-ID` for short reconnects; longer disconnects go through a fresh `register` (all prior leases already redelivered by the broker).

Policy Integration

Four hook points, each at a distinct lifecycle stage. Initially implemented as a `NoOpPolicyEngine` that always allows; real engine design deferred. Interface shipped in `packages/policy`:

Hook	Call Site	Input	Failure Mode
`checkRegistration`	Gateway, in register handler	identity, capability, scope	Registration rejected with code
`gatherPlanPolicy`	Planner, start of plan pipeline	goal, onBehalfOf, handlers	Opaque `PolicyContext` returned (NoOp → undefined)
`validatePlan`	Orchestrator, after LLM plan emission	plan, policyContext	Re-plan with violations, bounded retries, else terminal fail
`checkExecution`	Dispatcher, immediately before publish	task, plan, policyContext	Task fails with "policy changed since plan creation"

The `PolicyContext` returned by `gatherPlanPolicy` is **persisted alongside the plan**. `checkExecution` uses it as the at-plan-time snapshot for divergence detection — errors can name the specific policy that changed between plan time and execution time.

Data Flow: Hybrid Plan Execution

```
Client (desktop)                                Gateway                              Neuron (e.g.
     │                                             │                                         │
     │  1. Register(JWT, capabilities[])           │                                         │
     │────────────────────────────────────────────▶│                                         │
     │                                             │ verify JWT via Moku JWKS                │
     │                                             │ check namespace auth                    │
     │                                             │ policy.checkRegistration()              │
     │                                             │ declare per-user queues                 │
     │                                             │ bind consumer channels                  │
     │  2. registered                              │                                         │
     │◀────────────────────────────────────────────│                                         │
     │                                             │                                         │
     │  3. POST /plan { goal, handlers[] }         │                                         │
     │────────────────────────────────────────────▶│ gatherPlanPolicy()                      │
     │                                             │ filter handlers by policy               │
     │                                             │ LLM plan (with catalog + policy)        │
     │                                             │ validatePlan() + re-plan loop           │
     │                                             │ persist plan + catalog + policy snapshot │
     │                                             │                                         │
     │  4. plan created, workflow running          │                                         │
     │◀────────────────────────────────────────────│                                         │
     │                                             │ checkExecution() for ready task         │
     │                                             │ publish to tasks.user.{U}.{capability}  │
     │                                             │ (broker routes to bound consumer)       │
     │                                             │────────────────────────────────────────▶│ 5
     │                                             │                                         │
     │                                             │                                         │ v
     │                                             │                                         │ i
     │                                             │                                         │ (
     │                                             │                                         │
     │                                             │◀────────────────────────────────────────│ 6
     │                                             │ validate output against schema          │
     │                                             │ publish task.result → result processor  │
     │                                             │                                         │
     │                                             │ (result processor: apply patch,         │
     │                                             │  evaluate DAG, dispatch next task...)   │
```

Moku Boundary (Data Ownership)

BigBrain owns its own Postgres **schema** (not a separate physical database). Moku retains read access for reporting and is the schema authority (migrations reviewed through Moku's coordination process); no other service writes to BigBrain's tables. This solves the cross-service coordination problem at the source.

BigBrain ↔ Moku API consumption surface:

Capability	Cadence	Cached
JWT verification (JWKS fetch)	Rare (key rotation)	Yes, in-memory
User profile lookup	On-demand (UI, audit)	Yes, short TTL
Org membership	On auth	Yes, per-session
Policy config	On plan start	Yes, by version hash

Hot-path operations — task state transitions, lease renewals, heartbeats, context patches — hit the BigBrain schema directly. Moku is not in the hot path.

Key Design Decisions

Decision	Choice	Rationale
Execution model	Hybrid neuron-based routing by capability	Enables user-specific MCP execution (desktop) alongside server-side compute
Gateway placement	Mounted on `apps/api` for v1, isolated enough to extract	Faster to ship; scale-out path preserved
Transport	SSE (server→neuron) + HTTPS POST (neuron→server)	Enterprise-network friendly; WebSocket fails behind many corporate proxies
Auth	JWT from Moku (existing IDP integration); verified in-process via cached JWKS. Desktop: user token via RFC 8693 exchange. Enterprise: RFC 8628 enrollment → `client_credentials` short-lived tokens (see `NEURON_AUTH_SPEC.md`)	Reuses existing auth; no per-request Moku round-trip; long-lived artifact is a revocable credential, never a bearer token
Capability scoping	Per-advertisement (`any` or `onBehalfOf:userId`); auth token bounds `onBehalfOf`	Prevents user-scope spoofing; one neuron may mix scopes
Queue topology	Per-user queues lazily created, broker-reaped via `x-expires`	Natural per-user ordering; pending-state visibility; bounded growth
Lease semantics	Unacked AMQP messages + DB `assigned` state + lease expiry reaper	Restart-from-scratch on disconnect; no partial-progress preservation
Handler idempotency	Author contract (framework does not provide mid-task checkpointing)	Simpler framework; leverages capability-native idempotency keys
Concurrency	Per-capability advertisement → `basic.qos prefetch_count`	Broker handles backpressure; no cross-capability global cap in v1
Schema ownership	Per-plan catalog, snapshotted with plan; no global capability registry	Enables desktop-unique capabilities; avoids coordination bottleneck
Advertise authorization	Capability type = `{owner}/{package}/{name}` (naming convention); gateway admits against the org ledger (`org_capabilities.status`), entries created on first advertisement per the org's `capability_admission` setting (`NEURON_AUTH_SPEC.md` §2)	Zero-friction onboarding by default, explicit approval gate for regulated orgs; no pre-seeding; squatting contained by org-bound connect + org-partitioned queues
Wire protocol	`@workflow/neuron-protocol` package shared between gateway and SDK	Single source of truth for message shapes
Neuron SDK	`@workflow/neuron-sdk` ; used by built-in worker too	One implementation, one test surface
Policy integration	Four hooks (register / gather / validate / execute) with NoOp shim shipped first	Locks call sites without blocking on real engine design

Moku boundary	BigBrain owns its schema; Moku retains read for reporting	Coordination at schema-change time, not runtime

Failure Modes

Neuron disconnect mid-task

Broker redelivers unacked message. Reaper flips `assigned` → `pending`. Next consumer binding (same user, any device) re-leases. Handler must be idempotent.

User-scoped neuron offline at dispatch

Task sits in per-user queue (`pending` state in DB, visible in UI). `x-message-ttl` bounds the wait window; on expiry the task DLX's to the planner for re-plan or fail.

Capability collision attempt

Gateway rejects registration at the namespace-authorization check. Visible failure, surfaced to neuron with a specific error code.

Schema mismatch at execution

Neuron SDK pre-validates input before invoking handler; rejects with `schema-mismatch` nack. Output validated on ack; a mismatching output also nacks. Either surfaces through the result processor as a terminal task failure with a structured error.

Policy change mid-workflow

`checkExecution` compares at-plan-time snapshot to current policy. Divergence → task fails with a specific `policy-changed` error that names the rule and the plan-time version. The workflow can be re-planned against current policy if the user retriggers.

Gateway restart

Neurons notice SSE disconnect, attempt `register` via reconnection loop. All in-flight leases were unacked → broker redelivered them → reaper already moved them back to pending. Nothing to recover explicitly; connection re-establishment picks up naturally.

Open Items (Deferred)

- **Real policy engine design** — interface stable; engine design (rule language, classification propagation, obligations) deferred.
- **Neuron identity / service-token replacement** — settled and in flight, no longer open: RFC 8628 enrollment + `client_credentials` with per-client revocation (Moku side merged; BigBrain side =

HOL-2537 epic; contract in `NEURON_AUTH_SPEC.md`). Remaining deferred piece: `private_key_jwt` client authentication (HOL-2543).

- **Cross-capability global concurrency cap** — per-capability caps only in v1; add global if a real case emerges.
- **Browser-based neurons** — SDK is node-first; wire protocol stays browser-compatible for future.
- **Per-capability feature tags for version drift** — not in v1; add when needed.